# Solution: Shortest Word Distance

## Problem Statement

Given an array of strings `words` and two different strings that already exist in the array `word1` and `word2`, return the shortest distance between these two words in the list.

**Example 1:**

```
Input: words = ["the", "quick", "brown", "fox", "jumps", "over", "the",
"lazy", "dog"], word1 = "fox", word2 = "dog"
Output: 5
Explanation: The distance between "fox" and "dog" is 5 words.
```

**Example 2:**

```
Input: words = ["a", "c", "d", "b", "a"], word1 = "a", word2 = "b"
Output: 1
Explanation: The shortest distance between "a" and "b" is 1 word. Please
note that "a" appeared twice.
```

**Example 3:**

```
Input: words = ["a", "b", "c", "d", "e"], word1 = "a", word2 = "e"
Output: 4
Explanation: The distance between "a" and "e" is 4 words.
```

## Solution

To find the shortest distance between two given words, we can iterate through the list of words and use two pointers to track the positions of these words. Whenever we find one of these words in the list, we will update the word's position and calculate the distance from the other word. This way, we will keep track of the shortest distance between the two words.

Here's how the algorithm will work:

1. Initialize two variables `position1` and `position2` to store the positions of word1 and word2 in the words list, respectively. Initialize these variables to -1 which means that the words haven't been found yet.
2. Initialize a variable `shortestDistance` with the length of the words list as the initial value. This value will be updated later in the loop as the shortest distance is found.
3. Loop through the words list and for each word, check if it is `word1` or `word2`. If it is `word1`, update `position1` with the current index. If it is `word2`, update `position2` with the current index.
4. If both `position1` and `position2` have been updated, that means both `word1` and `word2` have been found in the words list. In this case, update the `shortestDistance` with the absolute difference of the positions of `word1` and `word2`.
5. Repeat steps 3 and 4 until the end of the words list is reached.
6. Return the final value of `shortestDistance` as the result.

## Code

Here is the code for this algorithm:

Java

```java
class Solution {
  public int shortestDistance(String[] words, String word1, String word2) {
    // Initialize the shortest distance with the length of the words list
    int shortestDistance = words.length;
    int position1 = -1, position2 = -1; // Initialize the positions of word1 and word2 w

    for (int i = 0; i < words.length; i++) {
      String word = words[i];
      if (word.equals(word1)) { // If the current word is word1, update the position1
        position1 = i;
      } else if (word.equals(word2)) { // If the current word is word2, update the posit
        position2 = i;
      }
      // If both the positions are updated, update the shortest distance
      if (position1 != -1 && position2 != -1) {
        shortestDistance = Math.min(shortestDistance, Math.abs(position1 - position2));
      }
    }
    return shortestDistance;
  }
```

Run Code Result | Stdout

You must run your code first

Execute

## Time and Space Complexity

1. **Time Complexity:** The method consists of a single loop that iterates through the `words` array. During each iteration, it performs constant-time operations, such as comparison and arithmetic operations. The loop runs once for each element in the `words` array.
   - Since the loop iterates through all elements in the `words` array, the time complexity is $O(n)$, where $n$ is the number of elements in the `words` array.
2. **Space Complexity:** The method uses a fixed number of integer variables (`position1`, `position2`, etc) and a temporary string variable (`word`). These variables consume a constant amount of space regardless of the input size.
   - Therefore, the space complexity is $O(1)$ as the amount of memory used does not scale with the size of the input array.

Please wait! Loading...